```
================================================================================

FILE: backend/app/main.py
--------------------------------------------------------------------------------
from __future__ import annotations

from fastapi import FastAPI, File, HTTPException, Query, UploadFile
from fastapi.middleware.cors import CORSMiddleware

from .config import get_settings
from .ml_service import ml_analyze_kus, ml_predict, ml_predict_nlp,
ml_status, ml_train, ml_train_nlp
from .schemas import (
    AiExplainRequest,
    MLNLPPredictRequest,
    MLNLPTrainRequest,
    MLPredictRequest,
    MLTrainRequest,
    MatchRequest,
)
from .services import explain_match_stub, list_courses,
list_institutions, match_from_pdf_bytes

app = FastAPI(title="Transfer Credit Match API", version="1.0.0")
settings = get_settings()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


@app.get("/health")
def health() -> dict:
    return {"status": "ok", "env": settings.app_env}


@app.get("/api/institutions")
def institutions() -> list[dict]:
    return list_institutions(settings)


@app.get("/api/courses")
def courses(institution_id: int | None = Query(default=None)) ->
```

```
list[dict]:
    return list_courses(settings, institution_id=institution_id)


@app.post("/api/match/transcript")
async def match_transcript(
    target_institution_id: int = Query(...),
    top_k: int = Query(default=3, ge=1, le=10),
    threshold: float = Query(default=0.55, ge=0.0, le=1.0),
    pdf_file: UploadFile = File(...),
) -> dict:
    if not pdf_file.filename or not
pdf_file.filename.lower().endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Only PDF
transcript uploads are supported.")
    content = await pdf_file.read()
    if not content:
        raise HTTPException(status_code=400, detail="Uploaded file is
empty.")
    try:
        return match_from_pdf_bytes(
            settings=settings,
            pdf_bytes=content,
            filename=pdf_file.filename,
            target_institution_id=target_institution_id,
            top_k=top_k,
            threshold=threshold,
        )
    except Exception as exc:  # noqa: BLE001
        raise HTTPException(status_code=500, detail=f"Failed to
process transcript: {exc}") from exc


@app.post("/api/ai/explain")
def explain(payload: AiExplainRequest) -> dict:
    return explain_match_stub(settings, payload.model_dump())


@app.get("/api/ml/status")
def model_status() -> dict:
    return ml_status()


@app.post("/api/ml/train")
def model_train(payload: MLTrainRequest) -> dict:
    return ml_train(
        samples=payload.samples,
        epochs=payload.epochs,
        learning_rate=payload.learning_rate,
        patience=payload.patience,
```

```
    )


@app.post("/api/ml/predict")
def model_predict(payload: MLPredictRequest) -> dict:
    if payload.source is not None and payload.target is not None:
        return ml_predict(source=payload.source,
target=payload.target)
    if payload.features is not None:
        return ml_predict(features=payload.features)
    raise HTTPException(status_code=400, detail="Provide source+target
or features.")


@app.post("/api/ml/train-nlp")
def model_train_nlp(payload: MLNLPTrainRequest) -> dict:
    return ml_train_nlp(
        csv_dir=payload.csv_dir,
        epochs=payload.epochs,
        learning_rate=payload.learning_rate,
        patience=payload.patience,
    )


@app.post("/api/ml/predict-nlp")
def model_predict_nlp(payload: MLNLPPredictRequest) -> dict:
    return ml_predict_nlp(source=payload.source,
target=payload.target, csv_dir=payload.csv_dir)


@app.post("/api/ml/analyze-kus")
def model_analyze_kus(payload: MLNLPPredictRequest) -> dict:
    return ml_analyze_kus(source=payload.source,
target=payload.target, csv_dir=payload.csv_dir)
```

```
================================================================================
==========

FILE: backend/app/config.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

import os
from dataclasses import dataclass
from pathlib import Path


@dataclass
class Settings:
```

```
    csv_root: Path
    rules_path: Path
    openai_api_key: str | None
    app_env: str


def get_settings() -> Settings:
    project_root = Path(__file__).resolve().parents[2]
    csv_root = Path(os.getenv("CSV_ROOT", project_root /
"csv_exports")).resolve()
    rules_path = Path(
        os.getenv("MATCHING_RULES_PATH", project_root / "config" /
"matching_rules.json")
    ).resolve()
    openai_api_key = os.getenv("OPENAI_API_KEY")
    app_env = os.getenv("APP_ENV", "dev")
    return Settings(
        csv_root=csv_root,
        rules_path=rules_path,
        openai_api_key=openai_api_key,
        app_env=app_env,
    )

================================================================================

FILE: backend/app/services.py
--------------------------------------------------------------------------------
from __future__ import annotations

import tempfile
from pathlib import Path
import sys

import pandas as pd

# Ensure project-root modules are importable when API runs from
backend/.
PROJECT_ROOT = Path(__file__).resolve().parents[2]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

from TranscriptPDF_reader.pdf_reader import extract_transcript_data
from engine.config import load_matching_rules
from engine.data_loader import load_data_bundle
from engine.pipeline import match_transcript_to_institution

from .config import Settings
```

```
def list_institutions(settings: Settings) -> list[dict]:
    institutions = pd.read_csv(settings.csv_root / "institutions.csv")
    return institutions.to_dict(orient="records")


def list_courses(settings: Settings, institution_id: int | None =
None) -> list[dict]:
    courses = pd.read_csv(settings.csv_root / "courses.csv")
    if institution_id is not None:
        courses = courses[courses["institution_id"] ==
int(institution_id)]
    return courses.to_dict(orient="records")


def match_from_pdf_bytes(
    settings: Settings,
    pdf_bytes: bytes,
    filename: str,
    target_institution_id: int,
    top_k: int,
    threshold: float,
) -> dict:
    suffix = Path(filename).suffix or ".pdf"
    with tempfile.NamedTemporaryFile(suffix=suffix, delete=True) as
tmp:
        tmp.write(pdf_bytes)
        tmp.flush()
        transcript_data = extract_transcript_data(tmp.name)

    bundle = load_data_bundle(settings.csv_root)
    rules = load_matching_rules(settings.rules_path)
    matches = match_transcript_to_institution(
        transcript_data=transcript_data,
        target_institution_id=target_institution_id,
        data_bundle=bundle,
        top_k=top_k,
        rules=rules,
        approval_threshold=threshold,
    )
    return {
        "student": {
            "name": transcript_data.get("name"),
            "institution": transcript_data.get("institution"),
            "major": transcript_data.get("major"),
            "year": transcript_data.get("year"),
        },
        "total_parsed_classes": len(transcript_data.get("classes",
[])),
        "matches": matches.to_dict(orient="records"),
```

```
    }


def explain_match_stub(settings: Settings, payload: dict) -> dict:
    if settings.openai_api_key:
        return {
            "mode": "api-key-present",
            "explanation": (
                "API key detected. Connect this endpoint to your
preferred LLM provider "
                "when you are ready for AI-generated advisor
explanations."
            ),
        }
    return {
        "mode": "no-api-key",
        "explanation": (
            f"{payload['source_course_name']} vs
{payload['target_course_name']} "
            f"scored {payload['score']:.2f} and was classified as
{payload['decision']}. "
            "Add OPENAI_API_KEY in backend/.env to enable AI
explanation generation later."
        ),
    }
```

```
==========

FILE: backend/app/schemas.py
----------------------------------------------------------------------
----------
from __future__ import annotations

from pydantic import BaseModel, Field


class MatchRequest(BaseModel):
    target_institution_id: int
    top_k: int = Field(default=3, ge=1, le=10)
    threshold: float = Field(default=0.55, ge=0.0, le=1.0)


class AiExplainRequest(BaseModel):
    source_course_name: str
    target_course_name: str
    score: float
    decision: str
```

```
class MLTrainRequest(BaseModel):
    samples: list | None = None
    epochs: int = 300
    learning_rate: float = 0.001
    patience: int = 50


class MLPredictRequest(BaseModel):
    source: list[float] | None = None
    target: list[float] | None = None
    features: list[float] | None = None


class MLNLPTrainRequest(BaseModel):
    csv_dir: str = "csv_exports"
    epochs: int = 300
    learning_rate: float = 0.001
    patience: int = 50


class MLNLPPredictRequest(BaseModel):
    source: int
    target: int
    csv_dir: str = "csv_exports"

================================================================================
==========

FILE: backend/app/ml_service.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[2]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

from ml import transfer_matcher_full as tm  # noqa: E402


def ml_train(samples=None, epochs: int = 300, learning_rate: float =
0.001, patience: int = 50):
    return tm.train_model(samples=samples, epochs=epochs,
learning_rate=learning_rate, patience=patience)


def ml_predict(source=None, target=None, features=None):
```

```
    return tm.predict(source=source, target=target, features=features)


def ml_status():
    return tm.get_status()


def ml_train_nlp(csv_dir: str, epochs: int = 300, learning_rate: float
= 0.001, patience: int = 50):
    return tm.train_nlp_model(csv_dir=csv_dir, epochs=epochs,
learning_rate=learning_rate, patience=patience)


def ml_predict_nlp(source: int, target: int, csv_dir: str):
    return tm.predict_nlp(src_course_id=source, tgt_course_id=target,
csv_dir=csv_dir)


def ml_analyze_kus(source: int, target: int, csv_dir: str):
    return tm.analyze_kus(src_course_id=source, tgt_course_id=target,
csv_dir=csv_dir)
```

```
==========

FILE: backend/run_api.py
----------------------------------------------------------------------
----------
from __future__ import annotations

import uvicorn


if __name__ == "__main__":
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000,
reload=True)

==========

FILE: backend/requirements.txt
----------------------------------------------------------------------
----------
fastapi
uvicorn
python-multipart
pydantic
pandas
pdfplumber
torch
```

```
scikit-learn
mysql-connector-python
requests

================================================================================
==========

FILE: backend/.env.example
--------------------------------------------------------------------------------
----------
APP_ENV=dev
CSV_ROOT=../csv_exports
MATCHING_RULES_PATH=../config/matching_rules.json

# Add your provider key later:
OPENAI_API_KEY=

================================================================================
==========

FILE: frontend/package.json
--------------------------------------------------------------------------------
----------
{
  "name": "transfer-credit-match-frontend",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^5.4.11"
  }
}

================================================================================
==========

FILE: frontend/index.html
--------------------------------------------------------------------------------
----------
<!doctype html>
```

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Transfer Credit Match</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

================================================================================

FILE: frontend/vite.config.js
--------------------------------------------------------------------------------
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173
  }
});

================================================================================

FILE: frontend/.env.example
--------------------------------------------------------------------------------
VITE_API_BASE_URL=http://localhost:8000

================================================================================

FILE: frontend/src/main.jsx
--------------------------------------------------------------------------------
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./styles.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
```

```
    <App />
  </React.StrictMode>
);

================================================================================

FILE: frontend/src/App.jsx
--------------------------------------------------------------------------------
import { useEffect, useMemo, useState } from "react";
import MatchTable from "./components/MatchTable";

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || "http://
localhost:8000";

async function apiGet(path) {
  const res = await fetch(`${API_BASE_URL}${path}`);
  if (!res.ok) {
    throw new Error(`GET ${path} failed: ${res.status}`);
  }
  return res.json();
}

async function apiMatchTranscript({ file, targetInstitutionId, topK,
threshold }) {
  const formData = new FormData();
  formData.append("pdf_file", file);

  const url = new URL(`${API_BASE_URL}/api/match/transcript`);
  url.searchParams.set("target_institution_id",
String(targetInstitutionId));
  url.searchParams.set("top_k", String(topK));
  url.searchParams.set("threshold", String(threshold));

  const res = await fetch(url, {
    method: "POST",
    body: formData
  });
  if (!res.ok) {
    const msg = await res.text();
    throw new Error(msg || `Upload failed with status ${res.status}`);
  }
  return res.json();
}

async function apiExplain(row) {
  const res = await fetch(`${API_BASE_URL}/api/ai/explain`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
```

```
    body: JSON.stringify({
      source_course_name: row.source_course_name,
      target_course_name: row.target_course_name,
      score: row.score,
      decision: row.decision
    })
  });
  if (!res.ok) {
    throw new Error("Explanation request failed.");
  }
  return res.json();
}

export default function App() {
  const [institutions, setInstitutions] = useState([]);
  const [targetInstitutionId, setTargetInstitutionId] = useState("");
  const [file, setFile] = useState(null);
  const [topK, setTopK] = useState(3);
  const [threshold, setThreshold] = useState(0.55);
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState(null);
  const [error, setError] = useState("");
  const [aiExplanation, setAiExplanation] = useState("");

  useEffect(() => {
    apiGet("/api/institutions")
      .then((data) => {
        setInstitutions(data);
        if (data.length) {
          setTargetInstitutionId(String(data[0].institution_id));
        }
      })
      .catch((err) => setError(err.message));
  }, []);

  const bestMatch = useMemo(() => {
    if (!result?.matches?.length) return null;
    return [...result.matches].sort((a, b) => b.score - a.score)[0];
  }, [result]);

  async function handleSubmit(e) {
    e.preventDefault();
    setError("");
    setAiExplanation("");
    if (!file) {
      setError("Please select a transcript PDF.");
      return;
    }
    if (!targetInstitutionId) {
      setError("Please select a destination institution.");
```

```
      return;
    }
    setLoading(true);
    try {
      const data = await apiMatchTranscript({
        file,
        targetInstitutionId,
        topK,
        threshold
      });
      setResult(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  async function handleExplainBestMatch() {
    if (!bestMatch) return;
    setAiExplanation("Loading explanation...");
    try {
      const data = await apiExplain(bestMatch);
      setAiExplanation(data.explanation);
    } catch (err) {
      setAiExplanation(`Could not get explanation: ${err.message}`);
    }
  }

  return (
    <main className="container">
      <h1>Transfer Credit Match</h1>
      <p className="subtitle">
        Upload transcript PDF -> parse courses -> compare with KU
mappings -> review likely equivalencies.
      </p>

      <form className="card" onSubmit={handleSubmit}>
        <div className="grid">
          <label>
            Destination Institution
            <select
              value={targetInstitutionId}
              onChange={(e) => setTargetInstitutionId(e.target.value)}
            >
              {institutions.map((inst) => (
                <option key={inst.institution_id}
value={inst.institution_id}>
                  {inst.name}
                </option>
```

```
              ))}
            </select>
          </label>
          <label>
            Top K Matches per Course
            <input
              type="number"
              min="1"
              max="10"
              value={topK}
              onChange={(e) => setTopK(Number(e.target.value))}
            />
          </label>
          <label>
            Auto-Equivalent Threshold
            <input
              type="number"
              step="0.01"
              min="0"
              max="1"
              value={threshold}
              onChange={(e) => setThreshold(Number(e.target.value))}
            />
          </label>
        </div>
        <label>
          Transcript PDF
          <input
            type="file"
            accept="application/pdf"
            onChange={(e) => setFile(e.target.files?.[0] || null)}
          />
        </label>
        <button type="submit" disabled={loading}>
          {loading ? "Processing..." : "Run Transfer Match"}
        </button>
      </form>

      {error ? <p className="error">{error}</p> : null}

      {result?.student ? (
        <section className="card">
          <h2>Transcript Summary</h2>
          <p><strong>Name:</strong> {result.student.name}</p>
          <p><strong>Institution:</strong>
{result.student.institution}</p>
          <p><strong>Major:</strong> {result.student.major}</p>
          <p><strong>Year:</strong> {result.student.year}</p>
          <p><strong>Parsed Classes:</strong>
{result.total_parsed_classes}</p>
```

```
        </section>
      ) : null}

      <section className="card">
        <h2>Match Results</h2>
        <MatchTable rows={result?.matches || []} />
      </section>

      <section className="card">
        <h2>Advisor Explanation (API key optional)</h2>
        <button onClick={handleExplainBestMatch} disabled={!bestMatch}
>
          Explain Best Match
        </button>
        <p className="mono">{aiExplanation || "No explanation
generated yet."}</p>
      </section>
    </main>
  );
}

==========================================================================
==========

FILE: frontend/src/components/MatchTable.jsx
--------------------------------------------------------------------------
----------
export default function MatchTable({ rows }) {
  if (!rows?.length) {
    return <p className="empty">No matches yet. Upload a transcript
PDF to begin.</p>;
  }

  return (
    <div className="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Source Course</th>
            <th>Target Course</th>
            <th>KU Overlap</th>
            <th>Score</th>
            <th>Decision</th>
          </tr>
        </thead>
        <tbody>
          {rows.map((row, idx) => (
            <tr key={`${row.source_course_code}-$
{row.target_course_code}-${idx}`}>
              <td>
```

```
                <strong>{row.source_course_code}</strong>
                <div>{row.source_course_name}</div>
              </td>
              <td>
                <strong>{row.target_course_code}</strong>
                <div>{row.target_course_name}</div>
              </td>
              <td>
                {row.ku_overlap_count} ({row.ku_overlap_ratio})
              </td>
              <td>{row.score}</td>
              <td>{row.decision}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

================================================================================

FILE: frontend/src/styles.css
--------------------------------------------------------------------------------
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Inter, system-ui, -apple-system, Segoe UI, Roboto,
sans-serif;
  background: #f4f7fb;
  color: #1a202c;
}

.container {
  max-width: 1100px;
  margin: 0 auto;
  padding: 24px;
}

.subtitle {
  color: #4a5568;
}

.card {
  background: #fff;
```

```
  border-radius: 12px;
  padding: 16px;
  margin: 16px 0;
  box-shadow: 0 4px 14px rgba(15, 23, 42, 0.08);
}

.grid {
  display: grid;
  gap: 12px;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
}

label {
  display: flex;
  flex-direction: column;
  gap: 6px;
  font-weight: 600;
  margin: 10px 0;
}

input,
select,
button {
  border: 1px solid #cbd5e0;
  border-radius: 8px;
  padding: 10px;
  font: inherit;
}

button {
  background: #2563eb;
  color: #fff;
  border: 0;
  cursor: pointer;
  font-weight: 600;
}

button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.error {
  color: #c53030;
  font-weight: 700;
}

.table-wrap {
  overflow: auto;
}
```

```
table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  border: 1px solid #e2e8f0;
  text-align: left;
  padding: 10px;
  vertical-align: top;
}

thead {
  background: #edf2f7;
}

.empty {
  color: #718096;
}

.mono {
  font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas,
monospace;
}

================================================================================

FILE: ml/transfer_matcher_full.py
--------------------------------------------------------------------------------
#!/usr/bin/env python3
"""
Transfer Course Matching System — v3 (Sprint III — Blocking Strategy)
================================================================================
Integrated into Transfer Credit Match project.
"""

import sys, os, json, re, argparse, datetime
from collections import defaultdict

import torch
import torch.nn as nn
import torch.optim as optim

MODEL_PATH = os.path.join(os.path.dirname(__file__),
"model_weights.pt")
MODEL_META_PATH = os.path.join(os.path.dirname(__file__),
```

```
"model_meta.json")

NUM_SUBJECTS = 6
FEATURE_SIZE = 19


def build_feature_vector(src, tgt):
    sc, sl, ss, slab = float(src[0]), int(src[1]), int(src[2]),
float(src[3])
    tc, tl, ts = float(tgt[0]), int(tgt[1]), int(tgt[2])
    src_oh = [1.0 if i == ss else 0.0 for i in range(NUM_SUBJECTS)]
    tgt_oh = [1.0 if i == ts else 0.0 for i in range(NUM_SUBJECTS)]
    return [
        1.0 if ss == ts else 0.0,
        float(sl - tl),
        1.0 if sl >= tl else 0.0,
        sc / max(tc, 0.5),
        1.0 if sc >= tc - 0.5 else 0.0,
        abs(sc - tc),
        slab,
        *src_oh,
        *tgt_oh,
    ]


def _legacy_to_pair(f8):
    return f8[:4], [f8[4], f8[5], f8[6], 0]


def build_blocking_index(course_list):
    index = defaultdict(list)
    for course in course_list:
        subject_code = int(course[2])
        index[subject_code].append(course)
    return dict(index)


class TransferMatcherNet(nn.Module):
    def __init__(self, input_size=FEATURE_SIZE):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_size, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 32),
            nn.BatchNorm1d(32),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(32, 16),
```

```
            nn.ReLU(),
            nn.Linear(16, 1),
            nn.Sigmoid(),
        )

    def forward(self, x):
        return self.network(x)


def _compute_norm(X):
    mean = X.mean(dim=0)
    std = X.std(dim=0)
    std[std < 1e-6] = 1.0
    return mean, std


def _normalize(X, mean, std):
    return (X - mean) / std


def get_sample_training_data():
    return [
        ([3, 0, 0, 0], [3, 0, 0, 0], 1),
        ([3, 1, 0, 0], [3, 1, 0, 0], 1),
        ([4, 1, 1, 1], [4, 1, 1, 0], 1),
        ([3, 0, 2, 0], [3, 0, 2, 0], 1),
        ([4, 2, 4, 0], [4, 2, 4, 0], 1),
        ([3, 0, 3, 0], [3, 0, 3, 0], 1),
        ([3, 1, 4, 0], [3, 1, 4, 0], 1),
        ([4, 0, 1, 1], [4, 0, 1, 0], 1),
        ([3, 3, 4, 0], [3, 3, 4, 0], 1),
        ([3, 0, 5, 0], [3, 0, 5, 0], 1),
        ([3, 1, 0, 0], [3, 0, 0, 0], 1),
        ([4, 2, 1, 1], [3, 1, 1, 0], 1),
        ([3, 3, 0, 0], [3, 2, 0, 0], 1),
        ([4, 3, 4, 0], [3, 2, 4, 0], 1),
        ([3, 2, 2, 0], [3, 1, 2, 0], 1),
        ([4, 1, 1, 1], [3, 0, 1, 0], 1),
        ([3, 0, 4, 0], [3, 1, 4, 0], 0),
        ([3, 1, 0, 0], [3, 2, 0, 0], 0),
        ([3, 2, 4, 0], [3, 3, 4, 0], 0),
        ([3, 0, 2, 0], [3, 1, 2, 0], 0),
        ([3, 0, 0, 0], [4, 0, 0, 0], 0),
        ([1, 0, 0, 0], [4, 0, 0, 0], 0),
        ([2, 1, 2, 0], [5, 1, 2, 0], 0),
        ([3, 0, 0, 0], [3, 0, 1, 0], 0),
        ([3, 0, 2, 0], [3, 0, 0, 0], 0),
        ([4, 1, 4, 0], [4, 1, 1, 0], 0),
        ([3, 0, 3, 0], [3, 0, 4, 0], 0),
        ([3, 1, 1, 1], [3, 1, 4, 0], 0),
```

```
        ([4, 2, 5, 0], [4, 2, 0, 0], 0),
        ([3, 1, 0, 0], [3, 1, 3, 0], 0),
        ([4, 0, 4, 0], [4, 0, 2, 0], 0),
        ([3, 0, 0, 0], [3, 0, 0, 1], 1),
        ([4, 2, 1, 0], [4, 2, 1, 1], 1),
        ([3.5, 0, 0, 0], [3, 0, 0, 0], 1),
        ([2.5, 0, 0, 0], [3, 0, 0, 0], 1),
    ]


def _samples_to_tensors(samples):
    fvs, labels = [], []
    for s in samples:
        if isinstance(s, dict):
            if "source" in s and "target" in s:
                fv = build_feature_vector(s["source"], s["target"])
            elif "features" in s:
                f = s["features"]
                fv = build_feature_vector(*_legacy_to_pair(f)) if
len(f) == 8 else f
            else:
                continue
            fvs.append(fv)
            labels.append(float(s["label"]))
        elif isinstance(s, (list, tuple)) and len(s) == 3:
            fvs.append(build_feature_vector(s[0], s[1]))
            labels.append(float(s[2]))
        elif isinstance(s, (list, tuple)) and len(s) == 2:
            f, lbl = s
            fv = build_feature_vector(*_legacy_to_pair(f)) if len(f)
== 8 else f
            fvs.append(fv)
            labels.append(float(lbl))
    return (
        torch.tensor(fvs, dtype=torch.float32),
        torch.tensor(labels, dtype=torch.float32).unsqueeze(1),
    )


def train_model(samples=None, epochs=300, learning_rate=0.001,
patience=50):
    if not samples:
        samples = get_sample_training_data()
    X, y = _samples_to_tensors(samples)
    n = len(X)
    feat_mean, feat_std = _compute_norm(X)
    X_norm = _normalize(X, feat_mean, feat_std)
    idx = torch.randperm(n)
    split = max(int(n * 0.8), 1)
    Xt, yt = X_norm[idx[:split]], y[idx[:split]]
```

```
    Xv, yv = X_norm[idx[split:]], y[idx[split:]]
    has_val = len(Xv) > 0
    model = TransferMatcherNet()
    criterion = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="min", patience=20, factor=0.5, min_lr=1e-6
    )
    best_val, pat_cnt, best_state, final_ep = float("inf"), 0, None, 0
    for ep in range(epochs):
        model.train()
        optimizer.zero_grad()
        loss = criterion(model(Xt), yt)
        loss.backward()
        optimizer.step()
        if has_val:
            model.eval()
            with torch.no_grad():
                vl = criterion(model(Xv), yv).item()
            scheduler.step(vl)
            if vl < best_val:
                best_val, pat_cnt = vl, 0
                best_state = {k: v.clone() for k, v in
model.state_dict().items()}
            else:
                pat_cnt += 1
                if pat_cnt >= patience:
                    break
        final_ep = ep + 1
    if best_state:
        model.load_state_dict(best_state)
    model.eval()
    with torch.no_grad():
        preds = model(X_norm)
        accuracy = ((preds >= 0.5).float() == y).float().mean().item()
        final_loss = criterion(preds, y).item()
    torch.save(model.state_dict(), MODEL_PATH)
    meta = {
        "isTrained": True,
        "lastTrainedAt":
datetime.datetime.now(datetime.timezone.utc).isoformat(),
        "trainingAccuracy": round(accuracy, 4),
        "epochs": final_ep,
        "finalLoss": round(final_loss, 6),
        "bestValLoss": round(best_val, 6) if has_val else None,
        "numSamples": n,
        "featureSize": FEATURE_SIZE,
        "normalization": {"mean": feat_mean.tolist(), "std":
feat_std.tolist()},
    }
```

```
    with open(MODEL_META_PATH, "w", encoding="utf-8") as f:
        json.dump(meta, f, indent=2)
    global _cached_model, _cached_norm
    _cached_model = _cached_norm = None
    return {
        "success": True,
        "epochs": final_ep,
        "finalLoss": round(final_loss, 6),
        "accuracy": round(accuracy, 4),
        "message": f"Training complete on {n} samples.",
    }


_cached_model = None
_cached_norm = None


def _load_model():
    global _cached_model, _cached_norm
    if _cached_model is not None:
        return _cached_model, _cached_norm
    if not os.path.exists(MODEL_PATH):
        raise RuntimeError("Model not trained. Run train_model
first.")
    m = TransferMatcherNet()
    m.load_state_dict(torch.load(MODEL_PATH, weights_only=True))
    m.eval()
    norm = (torch.zeros(FEATURE_SIZE), torch.ones(FEATURE_SIZE))
    if os.path.exists(MODEL_META_PATH):
        with open(MODEL_META_PATH, encoding="utf-8") as f:
            nd = json.load(f).get("normalization", {})
        if "mean" in nd and "std" in nd:
            norm = (
                torch.tensor(nd["mean"], dtype=torch.float32),
                torch.tensor(nd["std"], dtype=torch.float32),
            )
    _cached_model, _cached_norm = m, norm
    return m, norm


def _classify(prob):
    conf = "high" if prob >= 0.75 or prob <= 0.25 else "medium" if
prob >= 0.60 or prob <= 0.40 else "low"
    return {"matchProbability": round(prob, 4), "isMatch": prob >=
0.5, "confidence": conf}


def predict(features=None, source=None, target=None):
    model, (mean, std) = _load_model()
    if source is not None and target is not None:
```

```
        fv = build_feature_vector(source, target)
    elif features is not None:
        fv = build_feature_vector(*_legacy_to_pair(features)) if
len(features) == 8 else features
    else:
        raise ValueError("Provide (source, target) or features")
    X = _normalize(torch.tensor([fv], dtype=torch.float32), mean, std)
    with torch.no_grad():
        return _classify(model(X).item())


def predict_batch(pairs):
    model, (mean, std) = _load_model()
    fvs = []
    for p in pairs:
        if "source" in p and "target" in p:
            fvs.append(build_feature_vector(p["source"], p["target"]))
        elif "features" in p:
            f = p["features"]
            fvs.append(build_feature_vector(*_legacy_to_pair(f)) if
len(f) == 8 else f)
    if not fvs:
        return []
    X = _normalize(torch.tensor(fvs, dtype=torch.float32), mean, std)
    with torch.no_grad():
        probs = model(X).squeeze(-1).tolist()
    if isinstance(probs, float):
        probs = [probs]
    return [_classify(p) for p in probs]


def predict_batch_blocked(source_course, blocking_index):
    subject_code = int(source_course[2])
    candidates = blocking_index.get(subject_code, [])
    if not candidates:
        return []
    pairs = [{"source": source_course, "target": tgt} for tgt in
candidates if tgt != source_course]
    if not pairs:
        return []
    results = predict_batch(pairs)
    scored = []
    for pair, result in zip(pairs, results):
        scored.append(
            {
                "target": pair["target"],
                "matchProbability": result["matchProbability"],
                "isMatch": result["isMatch"],
                "confidence": result["confidence"],
            }
```

```
        )
    scored.sort(key=lambda x: x["matchProbability"], reverse=True)
    return scored


def find_matches_for_courses(source_courses, target_courses):
    blocking_index = build_blocking_index(target_courses)
    all_matches = {}
    for i, src in enumerate(source_courses):
        all_matches[i] = predict_batch_blocked(src, blocking_index)
    return all_matches


def get_status():
    if os.path.exists(MODEL_META_PATH):
        with open(MODEL_META_PATH, encoding="utf-8") as f:
            meta = json.load(f)
        meta["modelPath"] = MODEL_PATH
        return meta
    return {"isTrained": False, "modelPath": MODEL_PATH,
"featureSize": FEATURE_SIZE}


SUBJECT_MAP = {
    "math": 0,
    "mathematics": 0,
    "science": 1,
    "biology": 1,
    "chemistry": 1,
    "physics": 1,
    "english": 2,
    "writing": 2,
    "composition": 2,
    "history": 3,
    "social": 3,
    "cs": 4,
    "computer": 4,
    "programming": 4,
    "information": 4,
    "technology": 4,
}


def _encode_subject(s):
    if not isinstance(s, str):
        return 5
    s = s.lower().strip()
    for kw, code in SUBJECT_MAP.items():
        if kw in s:
            return code
```

```
    return 5


def _encode_level(val):
    if val is None:
        return 0
    nums = re.findall(r"\d+", str(val))
    if nums:
        n = int(nums[0])
        if n >= 400:
            return 3
        if n >= 300:
            return 2
        if n >= 200:
            return 1
    return 0


def _clean_text(text):
    if not isinstance(text, str):
        return ""
    text = text.lower()
    text = re.sub(r"[^a-z0-9\\s]", " ", text)
    return re.sub(r"\\s+", " ", text).strip()


NLP_FEATURE_SIZE = 14
NLP_MODEL_PATH = os.path.join(os.path.dirname(__file__),
"model_nlp_weights.pt")
NLP_MODEL_META_PATH = os.path.join(os.path.dirname(__file__),
"model_nlp_meta.json")
DEFAULT_CSV_DIR = os.path.join(os.path.dirname(__file__), "..",
"csv_exports")


def load_ku_data(csv_dir=DEFAULT_CSV_DIR):
    import pandas as pd
    csv_dir = os.path.abspath(csv_dir)
    paths = {
        "courses": os.path.join(csv_dir, "courses.csv"),
        "knowledge_units": os.path.join(csv_dir,
"knowledge_units.csv"),
        "course_ku": os.path.join(csv_dir, "course_ku.csv"),
    }
    for name, p in paths.items():
        if not os.path.exists(p):
            raise FileNotFoundError(f"Missing CSV '{name}': {p}")
    courses_df = pd.read_csv(paths["courses"])
    ku_df = pd.read_csv(paths["knowledge_units"])
    ck_df = pd.read_csv(paths["course_ku"])
```

```
    ku_map = {
        int(r["ku_id"]): {"name": str(r.get("ku_name", "")),
"description": str(r.get("ku_description", ""))}
        for _, r in ku_df.iterrows()
    }
    course_ku_map = defaultdict(set)
    for _, r in ck_df.iterrows():
        course_ku_map[int(r["course_id"])].add(int(r["ku_id"]))
    return courses_df, ku_map, course_ku_map


def build_ku_text_profiles(courses_df, ku_map, course_ku_map):
    profiles = {}
    for _, row in courses_df.iterrows():
        cid = int(row["course_id"])
        course_name = _clean_text(str(row.get("course_name", "")))
        ku_ids = course_ku_map.get(cid, set())
        ku_texts = []
        for kid in sorted(ku_ids):
            ku = ku_map.get(kid, {})
            ku_texts.extend([_clean_text(ku.get("name", "")),
_clean_text(ku.get("description", ""))])
        profiles[cid] = {"name": course_name, "ku_profile": " ".join(t
for t in ku_texts if t) or "unknown", "ku_ids": ku_ids}
    return profiles


def build_tfidf_vectors(profiles):
    from sklearn.feature_extraction.text import TfidfVectorizer
    course_ids = sorted(profiles.keys())
    ku_texts = [profiles[c]["ku_profile"] for c in course_ids]
    name_texts = [profiles[c]["name"] or "unknown" for c in
course_ids]
    ku_tfidf = TfidfVectorizer(min_df=1, ngram_range=(1, 2))
    name_tfidf = TfidfVectorizer(min_df=1, ngram_range=(1, 2))
    ku_matrix = ku_tfidf.fit_transform(ku_texts)
    name_matrix = name_tfidf.fit_transform(name_texts)
    return ku_tfidf, name_tfidf, ku_matrix, name_matrix, course_ids


def compute_nlp_features(src_cid, tgt_cid, profiles, ku_matrix,
name_matrix, course_ids, courses_df):
    from sklearn.metrics.pairwise import cosine_similarity
    cid_idx = {c: i for i, c in enumerate(course_ids)}

    def _get_row(cid):
        rows = courses_df[courses_df["course_id"] == cid]
        return rows.iloc[0] if not rows.empty else None

    def _credits(row):
```

```
        try:
            return float(row["credits"]) if row is not None else 3.0
        except Exception:
            return 3.0

    def _level(row):
        if row is None:
            return 0
        nums = re.findall(r"\d+", str(row.get("course_code", "")))
        if nums:
            n = int(nums[0])
            if n >= 400:
                return 3
            if n >= 300:
                return 2
            if n >= 200:
                return 1
        return 0

    def _subj(row):
        if row is None:
            return 4
        return _encode_subject(str(row.get("course_name", "")) + " " +
str(row.get("course_code", "")))

    sr, tr = _get_row(src_cid), _get_row(tgt_cid)
    src_credits, tgt_credits = _credits(sr), _credits(tr)
    src_kus = profiles.get(src_cid, {}).get("ku_ids", set())
    tgt_kus = profiles.get(tgt_cid, {}).get("ku_ids", set())
    union = src_kus | tgt_kus
    inter = src_kus & tgt_kus
    jaccard = len(inter) / len(union) if union else 0.0
    si, ti = cid_idx.get(src_cid), cid_idx.get(tgt_cid)
    if si is not None and ti is not None:
        tfidf_cos = float(cosine_similarity(ku_matrix[si],
ku_matrix[ti])[0, 0])
        name_cos = float(cosine_similarity(name_matrix[si],
name_matrix[ti])[0, 0])
    else:
        tfidf_cos = name_cos = 0.0
    return [
        src_credits,
        float(_level(sr)),
        float(_subj(sr)),
        0.0,
        tgt_credits,
        float(_level(tr)),
        float(_subj(tr)),
        abs(src_credits - tgt_credits),
        jaccard,
```

```
        float(len(src_kus)),
        float(len(tgt_kus)),
        abs(float(len(src_kus)) - float(len(tgt_kus))),
        tfidf_cos,
        name_cos,
    ]


class NLPTransferMatcherNet(nn.Module):
    def __init__(self, input_size=NLP_FEATURE_SIZE):
        super().__init__()
        self.features = nn.Sequential(
            nn.Linear(input_size, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(32, 16),
            nn.ReLU(),
        )
        self.head = nn.Linear(16, 1)

    def logits(self, x):
        return self.head(self.features(x))

    def forward(self, x):
        return torch.sigmoid(self.logits(x))


_cached_nlp_model = None
_cached_nlp_norm = None


def build_nlp_training_samples(csv_dir=DEFAULT_CSV_DIR):
    courses_df, ku_map, course_ku_map = load_ku_data(csv_dir)
    profiles = build_ku_text_profiles(courses_df, ku_map,
course_ku_map)
    _, _, ku_matrix, name_matrix, course_ids =
build_tfidf_vectors(profiles)
    samples = []
    for src_cid in course_ids:
        for tgt_cid in course_ids:
            if src_cid == tgt_cid:
                continue
            fv = compute_nlp_features(src_cid, tgt_cid, profiles,
ku_matrix, name_matrix, course_ids, courses_df)
            level_ok = fv[1] >= fv[5]
            credits_ok = fv[0] >= fv[4] - 0.5
            has_kus = fv[9] > 0 or fv[10] > 0
```

```
            label = 1.0 if (has_kus and fv[8] >= 0.1 and level_ok and
credits_ok) else 0.0
            samples.append({"features": fv, "label": label})
    return samples


def train_nlp_model(csv_dir=DEFAULT_CSV_DIR, epochs=300,
learning_rate=0.001, patience=50):
    samples = build_nlp_training_samples(csv_dir)
    if not samples:
        return {"success": False, "message": "No training samples
generated."}
    X = torch.tensor([s["features"] for s in samples],
dtype=torch.float32)
    y = torch.tensor([s["label"] for s in samples],
dtype=torch.float32).unsqueeze(1)
    n = len(X)
    feat_mean, feat_std = _compute_norm(X)
    X_norm = _normalize(X, feat_mean, feat_std)
    idx = torch.randperm(n)
    split = max(int(n * 0.8), 1)
    Xt, yt = X_norm[idx[:split]], y[idx[:split]]
    Xv, yv = X_norm[idx[split:]], y[idx[split:]]
    has_val = len(Xv) > 0
    model = NLPTransferMatcherNet()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode="min", patience=20, factor=0.5, min_lr=1e-6)
    n_pos = y.sum().item()
    n_neg = n - n_pos
    pos_weight = torch.tensor([n_neg / max(n_pos, 1)],
dtype=torch.float32)
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    best_val, pat_cnt, best_state, final_ep = float("inf"), 0, None, 0
    for ep in range(epochs):
        model.train()
        optimizer.zero_grad()
        loss = criterion(model.logits(Xt), yt)
        loss.backward()
        optimizer.step()
        if has_val:
            model.eval()
            with torch.no_grad():
                vl = criterion(model.logits(Xv), yv).item()
            scheduler.step(vl)
            if vl < best_val:
                best_val, pat_cnt = vl, 0
                best_state = {k: v.clone() for k, v in
model.state_dict().items()}
            else:
```

```
                pat_cnt += 1
                if pat_cnt >= patience:
                    break
        final_ep = ep + 1
    if best_state:
        model.load_state_dict(best_state)
    model.eval()
    bce = nn.BCELoss()
    with torch.no_grad():
        preds = model(X_norm)
        accuracy = ((preds >= 0.5).float() == y).float().mean().item()
        final_loss = bce(preds, y).item()
    torch.save(model.state_dict(), NLP_MODEL_PATH)
    meta = {
        "isTrained": True,
        "lastTrainedAt":
datetime.datetime.now(datetime.timezone.utc).isoformat(),
        "trainingAccuracy": round(accuracy, 4),
        "epochs": final_ep,
        "finalLoss": round(final_loss, 6),
        "bestValLoss": round(best_val, 6) if has_val else None,
        "numSamples": n,
        "featureSize": NLP_FEATURE_SIZE,
        "normalization": {"mean": feat_mean.tolist(), "std":
feat_std.tolist()},
    }
    with open(NLP_MODEL_META_PATH, "w", encoding="utf-8") as f:
        json.dump(meta, f, indent=2)
    global _cached_nlp_model, _cached_nlp_norm
    _cached_nlp_model = _cached_nlp_norm = None
    return {"success": True, "epochs": final_ep, "finalLoss":
round(final_loss, 6), "accuracy": round(accuracy, 4), "numSamples": n}


def _load_nlp_model():
    global _cached_nlp_model, _cached_nlp_norm
    if _cached_nlp_model is not None:
        return _cached_nlp_model, _cached_nlp_norm
    if not os.path.exists(NLP_MODEL_PATH):
        raise RuntimeError("NLP model not trained. Run train_nlp_model
first.")
    m = NLPTransferMatcherNet()
    m.load_state_dict(torch.load(NLP_MODEL_PATH, weights_only=True))
    m.eval()
    norm = (torch.zeros(NLP_FEATURE_SIZE),
torch.ones(NLP_FEATURE_SIZE))
    if os.path.exists(NLP_MODEL_META_PATH):
        with open(NLP_MODEL_META_PATH, encoding="utf-8") as f:
            nd = json.load(f).get("normalization", {})
        if "mean" in nd and "std" in nd:
```

```
            norm = (torch.tensor(nd["mean"], dtype=torch.float32),
torch.tensor(nd["std"], dtype=torch.float32))
    _cached_nlp_model, _cached_nlp_norm = m, norm
    return m, norm


def predict_nlp(src_course_id, tgt_course_id,
csv_dir=DEFAULT_CSV_DIR):
    model, (mean, std) = _load_nlp_model()
    courses_df, ku_map, course_ku_map = load_ku_data(csv_dir)
    profiles = build_ku_text_profiles(courses_df, ku_map,
course_ku_map)
    _, _, ku_matrix, name_matrix, course_ids =
build_tfidf_vectors(profiles)
    fv = compute_nlp_features(src_course_id, tgt_course_id, profiles,
ku_matrix, name_matrix, course_ids, courses_df)
    X = _normalize(torch.tensor([fv], dtype=torch.float32), mean, std)
    with torch.no_grad():
        prob = model(X).item()
    result = _classify(prob)
    result["sourceId"] = src_course_id
    result["targetId"] = tgt_course_id
    src_kus = profiles.get(src_course_id, {}).get("ku_ids", set())
    tgt_kus = profiles.get(tgt_course_id, {}).get("ku_ids", set())
    union = src_kus | tgt_kus
    inter = src_kus & tgt_kus
    result["kuOverlapRatio"] = round(len(inter) / len(union), 4) if
union else 0.0
    result["tfidfCosineSim"] = round(fv[12], 4)
    result["nameCosineSim"] = round(fv[13], 4)
    return result


def analyze_kus(src_course_id, tgt_course_id,
csv_dir=DEFAULT_CSV_DIR):
    from sklearn.metrics.pairwise import cosine_similarity
    courses_df, ku_map, course_ku_map = load_ku_data(csv_dir)
    profiles = build_ku_text_profiles(courses_df, ku_map,
course_ku_map)
    _, _, ku_matrix, name_matrix, course_ids =
build_tfidf_vectors(profiles)
    cid_idx = {c: i for i, c in enumerate(course_ids)}

    def _name(cid):
        rows = courses_df[courses_df["course_id"] == cid]
        return rows.iloc[0]["course_name"] if not rows.empty else
f"Course {cid}"

    src_kus = profiles.get(src_course_id, {}).get("ku_ids", set())
    tgt_kus = profiles.get(tgt_course_id, {}).get("ku_ids", set())
```

```
    shared = src_kus & tgt_kus
    src_only = src_kus - tgt_kus
    tgt_only = tgt_kus - src_kus
    union = src_kus | tgt_kus
    jaccard = len(shared) / len(union) if union else 0.0
    si, ti = cid_idx.get(src_course_id), cid_idx.get(tgt_course_id)
    tfidf_cos = float(cosine_similarity(ku_matrix[si], ku_matrix[ti])
[0, 0]) if si is not None and ti is not None else 0.0
    name_cos = float(cosine_similarity(name_matrix[si],
name_matrix[ti])[0, 0]) if si is not None and ti is not None else 0.0

    def _ku_list(ids):
        return [{"id": k, "name": ku_map.get(k, {}).get("name", "")}
for k in sorted(ids)]

    return {
        "sourceCourse": {"id": src_course_id, "name":
_name(src_course_id), "kuCount": len(src_kus), "kus":
_ku_list(src_kus)},
        "targetCourse": {"id": tgt_course_id, "name":
_name(tgt_course_id), "kuCount": len(tgt_kus), "kus":
_ku_list(tgt_kus)},
        "sharedKUs": _ku_list(shared),
        "srcOnlyKUs": _ku_list(src_only),
        "tgtOnlyKUs": _ku_list(tgt_only),
        "kuOverlapRatio": round(jaccard, 4),
        "tfidfCosineSim": round(tfidf_cos, 4),
        "nameCosineSim": round(name_cos, 4),
        "suggestedLabel": 1 if jaccard >= 0.2 else 0,
    }


def main():
    parser = argparse.ArgumentParser(prog="transfer_matcher_full.py")
    sub = parser.add_subparsers(dest="command")
    p_t = sub.add_parser("train")
    p_t.add_argument("--epochs", type=int, default=300)
    p_t.add_argument("--lr", type=float, default=0.001)
    p_t.add_argument("--patience", type=int, default=50)
    p_t.add_argument("--samples", type=str, default=None)
    p_p = sub.add_parser("predict")
    p_p.add_argument("payload", nargs="?", default="{}")
    sub.add_parser("status")
    p_tn = sub.add_parser("train_nlp")
    p_tn.add_argument("--csv-dir", default=DEFAULT_CSV_DIR)
    p_tn.add_argument("--epochs", type=int, default=300)
    p_tn.add_argument("--lr", type=float, default=0.001)
    p_tn.add_argument("--patience", type=int, default=50)
    p_pn = sub.add_parser("predict_nlp")
    p_pn.add_argument("--source", type=int, required=True)
```

```
    p_pn.add_argument("--target", type=int, required=True)
    p_pn.add_argument("--csv-dir", default=DEFAULT_CSV_DIR)
    p_ak = sub.add_parser("analyze_kus")
    p_ak.add_argument("--source", type=int, required=True)
    p_ak.add_argument("--target", type=int, required=True)
    p_ak.add_argument("--csv-dir", default=DEFAULT_CSV_DIR)
    args = parser.parse_args()
    if args.command == "train":
        samples = None
        if args.samples:
            with open(args.samples, encoding="utf-8") as f:
                samples = json.load(f)
        print(json.dumps(train_model(samples=samples,
epochs=args.epochs, learning_rate=args.lr, patience=args.patience),
indent=2))
    elif args.command == "predict":
        payload = json.loads(args.payload)
        src, tgt, feats = payload.get("source"),
payload.get("target"), payload.get("features")
        if src and tgt:
            result = predict(source=src, target=tgt)
        elif feats:
            result = predict(features=feats)
        else:
            raise SystemExit("Provide source/target or features.")
        print(json.dumps(result, indent=2))
    elif args.command == "status":
        print(json.dumps(get_status(), indent=2))
    elif args.command == "train_nlp":
        print(json.dumps(train_nlp_model(csv_dir=args.csv_dir,
epochs=args.epochs, learning_rate=args.lr, patience=args.patience),
indent=2))
    elif args.command == "predict_nlp":
        print(json.dumps(predict_nlp(args.source, args.target,
csv_dir=args.csv_dir), indent=2))
    elif args.command == "analyze_kus":
        print(json.dumps(analyze_kus(args.source, args.target,
csv_dir=args.csv_dir), indent=2))
    else:
        parser.print_help()


if __name__ == "__main__":
    main()

================================================================================
==========

FILE: engine/__init__.py
--------------------------------------------------------------------------------
```

```
----------
"""Transfer Credit Match data-science engine."""

from .data_loader import load_data_bundle
from .matcher import compute_course_match
from .pipeline import match_transcript_to_institution
from .workflow import apply_match_decision,
create_transfer_request_and_match

__all__ = [
    "load_data_bundle",
    "compute_course_match",
    "match_transcript_to_institution",
    "create_transfer_request_and_match",
    "apply_match_decision",
]

================================================================================
==========

FILE: engine/data_loader.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import pandas as pd


@dataclass
class DataBundle:
    courses: pd.DataFrame
    course_ku: pd.DataFrame
    knowledge_units: pd.DataFrame
    institutions: pd.DataFrame
    programs: pd.DataFrame
    transfer_requests: pd.DataFrame
    students: pd.DataFrame
    course_syllabus: pd.DataFrame | None = None


def _read_csv(root: Path, name: str) -> pd.DataFrame:
    path = root / name
    if not path.exists():
        raise FileNotFoundError(f"Required dataset is missing:
{path}")
    return pd.read_csv(path)
```

```
def load_data_bundle(csv_root: str | Path) -> DataBundle:
    csv_root_path = Path(csv_root).resolve()
    syllabus_path = csv_root_path / "course_syllabus.csv"
    syllabus_df = pd.read_csv(syllabus_path) if syllabus_path.exists()
else None
    return DataBundle(
        courses=_read_csv(csv_root_path, "courses.csv"),
        course_ku=_read_csv(csv_root_path, "course_ku.csv"),
        knowledge_units=_read_csv(csv_root_path,
"knowledge_units.csv"),
        institutions=_read_csv(csv_root_path, "institutions.csv"),
        programs=_read_csv(csv_root_path, "programs.csv"),
        transfer_requests=_read_csv(csv_root_path,
"transfer_requests.csv"),
        students=_read_csv(csv_root_path, "students.csv"),
        course_syllabus=syllabus_df,
    )

==========================================================================
==========

FILE: engine/matcher.py
--------------------------------------------------------------------------
----------
from __future__ import annotations

from dataclasses import dataclass
from difflib import SequenceMatcher
from typing import Iterable

from .text_similarity import cosine_token_similarity,
jaccard_similarity


@dataclass
class MatchResult:
    source_course_code: str
    source_course_name: str
    target_course_code: str
    target_course_name: str
    source_credits: float
    target_credits: float
    ku_overlap_count: int
    ku_overlap_ratio: float
    title_similarity: float
    syllabus_similarity: float
    score: float
    decision: str
```

```
def _safe_ratio(a: float, b: float) -> float:
    if a <= 0 or b <= 0:
        return 0.0
    return min(a, b) / max(a, b)


def _title_similarity(a: str, b: str) -> float:
    return SequenceMatcher(None, a.lower().strip(),
b.lower().strip()).ratio()


def compute_course_match(
    source_course: dict,
    target_course: dict,
    source_ku_ids: Iterable[int],
    target_ku_ids: Iterable[int],
    source_syllabus: str = "",
    target_syllabus: str = "",
    rule: dict | None = None,
    approval_threshold: float = 0.55,
) -> MatchResult:
    source_set = set(source_ku_ids)
    target_set = set(target_ku_ids)
    overlap = source_set & target_set

    union_size = len(source_set | target_set)
    overlap_ratio = len(overlap) / union_size if union_size else 0.0
    title_sim = _title_similarity(source_course["course_name"],
target_course["course_name"])
    credit_ratio = _safe_ratio(float(source_course["credits"]),
float(target_course["credits"]))
    syllabus_sim = (
        0.5 * cosine_token_similarity(source_syllabus,
target_syllabus)
        + 0.5 * jaccard_similarity(source_syllabus, target_syllabus)
    )

    effective_rule = rule or {}
    weights = effective_rule.get(
        "weights",
        {"ku_overlap": 0.60, "title_similarity": 0.20, "credit_ratio":
0.10, "syllabus_similarity": 0.10},
    )
    min_ku_overlap = float(effective_rule.get("min_ku_overlap_ratio",
0.0))
    min_credit_ratio = float(effective_rule.get("min_credit_ratio",
0.0))
    effective_threshold =
float(effective_rule.get("approval_threshold", approval_threshold))
```

```
    score = (
        float(weights.get("ku_overlap", 0.0)) * overlap_ratio
        + float(weights.get("title_similarity", 0.0)) * title_sim
        + float(weights.get("credit_ratio", 0.0)) * credit_ratio
        + float(weights.get("syllabus_similarity", 0.0)) *
syllabus_sim
    )

    if overlap_ratio == 0:
        decision = "Not Equivalent"
    elif overlap_ratio < min_ku_overlap or credit_ratio <
min_credit_ratio:
        decision = "Review"
    elif score >= effective_threshold:
        decision = "Equivalent"
    else:
        decision = "Review"

    return MatchResult(
        source_course_code=source_course["course_code"],
        source_course_name=source_course["course_name"],
        target_course_code=target_course["course_code"],
        target_course_name=target_course["course_name"],
        source_credits=float(source_course["credits"]),
        target_credits=float(target_course["credits"]),
        ku_overlap_count=len(overlap),
        ku_overlap_ratio=round(overlap_ratio, 4),
        title_similarity=round(title_sim, 4),
        syllabus_similarity=round(syllabus_sim, 4),
        score=round(score, 4),
        decision=decision,
    )

================================================================
==========

FILE: engine/pipeline.py
----------------------------------------------------------------
----------
from __future__ import annotations

from dataclasses import asdict

import pandas as pd

from .config import resolve_rule
from .data_loader import DataBundle
from .matcher import compute_course_match
```

```
def _index_course_ku(course_ku_df: pd.DataFrame) -> dict[int,
set[int]]:
    grouped = course_ku_df.groupby("course_id")["ku_id"].apply(set)
    return grouped.to_dict()


def _normalize_transcript_course_code(value: str) -> str:
    return " ".join(str(value).strip().split())


def match_transcript_to_institution(
    transcript_data: dict,
    target_institution_id: int,
    data_bundle: DataBundle,
    top_k: int = 3,
    rules: dict | None = None,
    approval_threshold: float = 0.55,
) -> pd.DataFrame:
    courses = data_bundle.courses.copy()
    ku_index = _index_course_ku(data_bundle.course_ku)

    target_courses = courses[courses["institution_id"] ==
int(target_institution_id)]
    syllabus_index = {}
    if data_bundle.course_syllabus is not None and not
data_bundle.course_syllabus.empty:
        syllabus_index = dict(
            zip(
                data_bundle.course_syllabus["course_id"].astype(int),

data_bundle.course_syllabus["syllabus_text"].astype(str),
            )
        )
    rule = resolve_rule(rules or {"global_default": {}},
int(target_institution_id))

    if target_courses.empty:
        raise ValueError(f"No target courses found for
institution_id={target_institution_id}")

    transcript_classes = transcript_data.get("classes", [])
    if not transcript_classes:
        return pd.DataFrame()

    transcript_index = {
        _normalize_transcript_course_code(item["course"]): item for
item in transcript_classes
    }
```

```
    matches = []
    for source_code, source_item in transcript_index.items():
        source_lookup = courses[courses["course_code"] == source_code]
        if source_lookup.empty:
            continue
        source_course = source_lookup.iloc[0].to_dict()
        source_ku = ku_index.get(int(source_course["course_id"]),
set())

        scored_candidates = []
        for _, target_course in target_courses.iterrows():
            target_dict = target_course.to_dict()
            target_ku = ku_index.get(int(target_dict["course_id"]),
set())
            result = compute_course_match(
                source_course=source_course,
                target_course=target_dict,
                source_ku_ids=source_ku,
                target_ku_ids=target_ku,

source_syllabus=syllabus_index.get(int(source_course["course_id"]),
""),

target_syllabus=syllabus_index.get(int(target_dict["course_id"]), ""),
                rule=rule,
                approval_threshold=approval_threshold,
            )
            scored_candidates.append(result)

        for result in sorted(scored_candidates, key=lambda x: x.score,
reverse=True)[:top_k]:
            row = asdict(result)
            row["source_term"] = source_item.get("term", "Unknown")
            row["source_grade"] = source_item.get("grade", "Unknown")
            matches.append(row)

    return pd.DataFrame(matches).sort_values(
        by=["source_course_code", "score"], ascending=[True, False]
    )

==========================================================================
==========

FILE: engine/workflow.py
--------------------------------------------------------------------------
----------
from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime
```

```
from pathlib import Path

import pandas as pd


def _now_str() -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")


def _csv_path(csv_root: str | Path, file_name: str) -> Path:
    return Path(csv_root).resolve() / file_name


def _next_id(df: pd.DataFrame, id_col: str) -> int:
    if df.empty:
        return 1
    return int(df[id_col].max()) + 1


@dataclass
class RequestResult:
    request_id: int
    match_id: int
    course_to: int
    status: str


def create_transfer_request_and_match(
    csv_root: str | Path,
    student_id: int,
    institution_from: int,
    institution_to: int,
    course_from: int,
    course_to: int,
) -> RequestResult:
    root = Path(csv_root).resolve()
    requests_path = _csv_path(root, "transfer_requests.csv")
    match_path = _csv_path(root, "course_match.csv")

    requests_df = pd.read_csv(requests_path)
    match_df = pd.read_csv(match_path)

    request_id = _next_id(requests_df, "request_id")
    match_id = _next_id(match_df, "match_id")
    now = _now_str()

    req_row = pd.DataFrame(
        [
            {
                "request_id": request_id,
```

```
                "student_id": int(student_id),
                "course_from": int(course_from),
                "course_to": int(course_to),
                "institution_from": int(institution_from),
                "institution_to": int(institution_to),
                "status": "Pending",
                "request_date": now,
            }
        ]
    )
    match_row = pd.DataFrame(
        [
            {
                "match_id": match_id,
                "institution_from": int(institution_from),
                "institution_to": int(institution_to),
                "course_from": int(course_from),
                "course_to": int(course_to),
                "match_status": "Pending",
                "created_at": now,
            }
        ]
    )

    pd.concat([requests_df, req_row],
ignore_index=True).to_csv(requests_path, index=False)
    pd.concat([match_df, match_row],
ignore_index=True).to_csv(match_path, index=False)

    return RequestResult(
        request_id=request_id,
        match_id=match_id,
        course_to=int(course_to),
        status="Pending",
    )


def apply_match_decision(
    csv_root: str | Path,
    match_id: int,
    changed_by: int,
    decision: str,
) -> None:
    if decision not in {"Approved", "Denied"}:
        raise ValueError("decision must be one of: Approved, Denied")

    root = Path(csv_root).resolve()
    match_path = _csv_path(root, "course_match.csv")
    history_path = _csv_path(root, "match_history.csv")
```

```
    match_df = pd.read_csv(match_path)
    history_df = pd.read_csv(history_path)

    match_mask = match_df["match_id"] == int(match_id)
    if not match_mask.any():
        raise ValueError(f"match_id={match_id} not found")

    match_df.loc[match_mask, "match_status"] = decision
    match_df.to_csv(match_path, index=False)

    log_id = _next_id(history_df, "log_id")
    hist_row = pd.DataFrame(
        [
            {
                "log_id": log_id,
                "match_id": int(match_id),
                "changed_by": int(changed_by),
                "change_type": decision,
                "change_date": _now_str(),
            }
        ]
    )
    pd.concat([history_df, hist_row],
ignore_index=True).to_csv(history_path, index=False)

================================================================================
==========

FILE: engine/config.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

import json
from pathlib import Path


DEFAULT_RULES = {
    "global_default": {
        "approval_threshold": 0.55,
        "min_ku_overlap_ratio": 0.15,
        "min_credit_ratio": 0.66,
        "weights": {
            "ku_overlap": 0.60,
            "title_similarity": 0.20,
            "credit_ratio": 0.10,
            "syllabus_similarity": 0.10,
        },
    },
    "institution_rules": {},
```

```
}


def load_matching_rules(path: str | Path | None) -> dict:
    if path is None:
        return DEFAULT_RULES
    rules_path = Path(path)
    if not rules_path.exists():
        return DEFAULT_RULES
    with rules_path.open("r", encoding="utf-8") as f:
        loaded = json.load(f)
    merged = DEFAULT_RULES.copy()
    merged.update(loaded)
    return merged


def resolve_rule(rules: dict, institution_id: int) -> dict:
    specific = rules.get("institution_rules",
{}).get(str(institution_id), {})
    base = dict(rules.get("global_default", {}))
    base_weights = dict(base.get("weights", {}))
    base_weights.update(specific.get("weights", {}))
    base["weights"] = base_weights
    for key, value in specific.items():
        if key != "weights":
            base[key] = value
    return base

================================================================================
==========

FILE: engine/text_similarity.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

import re
from collections import Counter


TOKEN_PATTERN = re.compile(r"[a-zA-Z0-9]+")


def tokenize(text: str) -> list[str]:
    return TOKEN_PATTERN.findall((text or "").lower())


def jaccard_similarity(a: str, b: str) -> float:
    set_a = set(tokenize(a))
    set_b = set(tokenize(b))
```

```
    if not set_a and not set_b:
        return 0.0
    union = set_a | set_b
    if not union:
        return 0.0
    return len(set_a & set_b) / len(union)


def cosine_token_similarity(a: str, b: str) -> float:
    tok_a = Counter(tokenize(a))
    tok_b = Counter(tokenize(b))
    if not tok_a or not tok_b:
        return 0.0

    dot = sum(tok_a[token] * tok_b[token] for token in set(tok_a) &
set(tok_b))
    mag_a = sum(v * v for v in tok_a.values()) ** 0.5
    mag_b = sum(v * v for v in tok_b.values()) ** 0.5
    if mag_a == 0.0 or mag_b == 0.0:
        return 0.0
    return dot / (mag_a * mag_b)

================================================================================
==========

FILE: scripts/run_transfer_match.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

import argparse
from pathlib import Path

from engine import load_data_bundle, match_transcript_to_institution
from engine.config import load_matching_rules
from TranscriptPDF_reader.pdf_reader import extract_transcript_data


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Run transcript-to-course transfer matching
against seeded CSV data."
    )
    parser.add_argument("--pdf", required=True, help="Path to
transcript PDF")
    parser.add_argument(
        "--csv-root",
        default="csv_exports",
        help="Directory containing seeded CSV exports (default:
csv_exports)",
```

```
    )
    parser.add_argument(
        "--target-institution-id",
        required=True,
        type=int,
        help="Destination institution_id from institutions.csv",
    )
    parser.add_argument(
        "--top-k",
        type=int,
        default=3,
        help="Top target matches to keep for each source course",
    )
    parser.add_argument(
        "--threshold",
        type=float,
        default=0.55,
        help="Score threshold for auto-equivalent decision",
    )
    parser.add_argument(
        "--output",
        default="match_output.csv",
        help="Output CSV path for match results",
    )
    parser.add_argument(
        "--rules",
        default="config/matching_rules.json",
        help="Matching rules JSON path (default: config/
matching_rules.json)",
    )
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    pdf_path = Path(args.pdf).resolve()
    csv_root = Path(args.csv_root).resolve()
    output_path = Path(args.output).resolve()
    rules = load_matching_rules(args.rules)

    transcript_data = extract_transcript_data(str(pdf_path))
    data_bundle = load_data_bundle(csv_root)
    result_df = match_transcript_to_institution(
        transcript_data=transcript_data,
        target_institution_id=args.target_institution_id,
        data_bundle=data_bundle,
        top_k=args.top_k,
        rules=rules,
        approval_threshold=args.threshold,
    )
```

```
    if result_df.empty:
        print("No matches produced. Check transcript course codes
against courses.csv.")
        return

    result_df.to_csv(output_path, index=False)
    print(f"Saved {len(result_df)} match rows to: {output_path}")
    print(result_df.head(10).to_string(index=False))


if __name__ == "__main__":
    main()

================================================================================
==========

FILE: scripts/build_sqlite.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

import argparse
import sqlite3
from pathlib import Path

import pandas as pd


TABLE_FILES = {
    "institutions": "institutions.csv",
    "programs": "programs.csv",
    "courses": "courses.csv",
    "knowledge_units": "knowledge_units.csv",
    "course_ku": "course_ku.csv",
    "students": "students.csv",
    "transfer_requests": "transfer_requests.csv",
    "course_match": "course_match.csv",
    "match_history": "match_history.csv",
    "admins": "admins.csv",
    "directors": "directors.csv",
    "users": "users.csv",
}


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Build SQLite DB from
Data_Science CSV exports.")
    parser.add_argument(
        "--csv-root", default="csv_exports", help="Directory
```

```
containing input CSV files"
    )
    parser.add_argument(
        "--db-path", default="transfer_credit_match.db", help="Output
SQLite database path"
    )
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    csv_root = Path(args.csv_root).resolve()
    db_path = Path(args.db_path).resolve()

    with sqlite3.connect(db_path) as conn:
        for table_name, csv_name in TABLE_FILES.items():
            csv_path = csv_root / csv_name
            frame = pd.read_csv(csv_path)
            frame.to_sql(table_name, conn, if_exists="replace",
index=False)
            print(f"Loaded {table_name:<18} rows={len(frame):>4} from
{csv_name}")

    print(f"\nSQLite database built at: {db_path}")


if __name__ == "__main__":
    main()

================================================================================

FILE: scripts/validate_data.py
--------------------------------------------------------------------------------
from __future__ import annotations

import argparse
from pathlib import Path

import pandas as pd


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Validate
Data_Science CSV integrity.")
    parser.add_argument("--csv-root", default="csv_exports", help="CSV
directory")
    return parser.parse_args()
```

```
def _load(root: Path, name: str) -> pd.DataFrame:
    return pd.read_csv(root / name)


def main() -> None:
    args = parse_args()
    root = Path(args.csv_root).resolve()

    institutions = _load(root, "institutions.csv")
    programs = _load(root, "programs.csv")
    courses = _load(root, "courses.csv")
    ku = _load(root, "knowledge_units.csv")
    course_ku = _load(root, "course_ku.csv")
    students = _load(root, "students.csv")
    requests = _load(root, "transfer_requests.csv")
    matches = _load(root, "course_match.csv")

    errors: list[str] = []

    if institutions["institution_id"].duplicated().any():
        errors.append("Duplicate institution_id values in
institutions.csv")
    if courses["course_id"].duplicated().any():
        errors.append("Duplicate course_id values in courses.csv")
    if ku["ku_id"].duplicated().any():
        errors.append("Duplicate ku_id values in knowledge_units.csv")

    invalid_program_institutions = set(programs["institution_id"]) -
set(institutions["institution_id"])
    if invalid_program_institutions:
        errors.append(f"programs.csv has unknown institution_id
values: {sorted(invalid_program_institutions)}")

    invalid_course_programs = set(courses["program_id"]) -
set(programs["program_id"])
    if invalid_course_programs:
        errors.append(f"courses.csv has unknown program_id values:
{sorted(invalid_course_programs)}")

    invalid_course_ku_course = set(course_ku["course_id"]) -
set(courses["course_id"])
    if invalid_course_ku_course:
        errors.append(f"course_ku.csv has unknown course_id values:
{sorted(invalid_course_ku_course)}")

    invalid_course_ku_ku = set(course_ku["ku_id"]) - set(ku["ku_id"])
    if invalid_course_ku_ku:
        errors.append(f"course_ku.csv has unknown ku_id values:
{sorted(invalid_course_ku_ku)}")
```

```
    invalid_student_institutions = set(students["institution_id"]) -
set(institutions["institution_id"])
    if invalid_student_institutions:
        errors.append(f"students.csv has unknown institution_id
values: {sorted(invalid_student_institutions)}")

    invalid_request_courses = (set(requests["course_from"]) |
set(requests["course_to"])) - set(courses["course_id"])
    if invalid_request_courses:
        errors.append(f"transfer_requests.csv has unknown course ids:
{sorted(invalid_request_courses)}")

    invalid_match_courses = (set(matches["course_from"]) |
set(matches["course_to"])) - set(courses["course_id"])
    if invalid_match_courses:
        errors.append(f"course_match.csv has unknown course ids:
{sorted(invalid_match_courses)}")

    if errors:
        print("VALIDATION FAILED")
        for err in errors:
            print(f"- {err}")
        raise SystemExit(1)

    print("VALIDATION PASSED")
    print(f"institutions={len(institutions)} programs={len(programs)}
courses={len(courses)}")
    print(f"knowledge_units={len(ku)}
course_ku_links={len(course_ku)}")
    print(f"students={len(students)} transfer_requests={len(requests)}
course_match={len(matches)}")


if __name__ == "__main__":
    main()

================================================================================
==========

FILE: scripts/workflow_cli.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

import argparse
from pathlib import Path

import pandas as pd
```

```
from engine import apply_match_decision,
create_transfer_request_and_match


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="Transfer workflow
CLI (submit/review/report).")
    sub = parser.add_subparsers(dest="cmd", required=True)

    submit = sub.add_parser("submit", help="Submit transfer request
and create pending match")
    submit.add_argument("--csv-root", default="csv_exports")
    submit.add_argument("--student-id", required=True, type=int)
    submit.add_argument("--institution-from", required=True, type=int)
    submit.add_argument("--institution-to", required=True, type=int)
    submit.add_argument("--course-from", required=True, type=int)
    submit.add_argument("--course-to", required=True, type=int)

    review = sub.add_parser("review", help="Apply Approved/Denied
decision to a match")
    review.add_argument("--csv-root", default="csv_exports")
    review.add_argument("--match-id", required=True, type=int)
    review.add_argument("--changed-by", required=True, type=int)
    review.add_argument("--decision", required=True,
choices=["Approved", "Denied"])

    report = sub.add_parser("report", help="Export student match
report")
    report.add_argument("--csv-root", default="csv_exports")
    report.add_argument("--student-id", required=True, type=int)
    report.add_argument("--output",
default="student_transfer_report.csv")

    return parser


def submit_cmd(args: argparse.Namespace) -> None:
    result = create_transfer_request_and_match(
        csv_root=args.csv_root,
        student_id=args.student_id,
        institution_from=args.institution_from,
        institution_to=args.institution_to,
        course_from=args.course_from,
        course_to=args.course_to,
    )
    print(
        f"Submitted request_id={result.request_id}
match_id={result.match_id} "
        f"course_to={result.course_to} status={result.status}"
    )
```

```
def review_cmd(args: argparse.Namespace) -> None:
    apply_match_decision(
        csv_root=args.csv_root,
        match_id=args.match_id,
        changed_by=args.changed_by,
        decision=args.decision,
    )
    print(f"Updated match_id={args.match_id} to {args.decision} and
wrote match_history log.")


def report_cmd(args: argparse.Namespace) -> None:
    root = Path(args.csv_root).resolve()
    requests = pd.read_csv(root / "transfer_requests.csv")
    matches = pd.read_csv(root / "course_match.csv")
    courses = pd.read_csv(root / "courses.csv")

    selected = requests[requests["student_id"] ==
int(args.student_id)].copy()
    if selected.empty:
        print(f"No transfer requests found for
student_id={args.student_id}")
        return

    merged = selected.merge(
        matches,
        left_on=["course_from", "course_to", "institution_from",
"institution_to"],
        right_on=["course_from", "course_to", "institution_from",
"institution_to"],
        how="left",
        suffixes=("_request", "_match"),
    )

    merged = merged.merge(
        courses[["course_id", "course_code", "course_name"]],
        left_on="course_from",
        right_on="course_id",
        how="left",
    ).rename(columns={"course_code": "course_from_code",
"course_name": "course_from_name"})
    merged = merged.merge(
        courses[["course_id", "course_code", "course_name"]],
        left_on="course_to",
        right_on="course_id",
        how="left",
        suffixes=("", "_to"),
    ).rename(columns={"course_code": "course_to_code", "course_name":
```

```
"course_to_name"})

    cols = [
        "request_id",
        "student_id",
        "institution_from",
        "institution_to",
        "course_from_code",
        "course_from_name",
        "course_to_code",
        "course_to_name",
        "status",
        "match_status",
        "request_date",
        "created_at",
    ]
    output = Path(args.output).resolve()
    merged[cols].to_csv(output, index=False)
    print(f"Saved report rows={len(merged)} to {output}")


def main() -> None:
    args = build_parser().parse_args()
    if args.cmd == "submit":
        submit_cmd(args)
    elif args.cmd == "review":
        review_cmd(args)
    elif args.cmd == "report":
        report_cmd(args)


if __name__ == "__main__":
    main()

================================================================================
==========

FILE: config/matching_rules.json
--------------------------------------------------------------------------------
----------
{
  "global_default": {
    "approval_threshold": 0.55,
    "min_ku_overlap_ratio": 0.15,
    "min_credit_ratio": 0.66,
    "weights": {
      "ku_overlap": 0.6,
      "title_similarity": 0.2,
      "credit_ratio": 0.1,
      "syllabus_similarity": 0.1
```

```
    }
  },
  "institution_rules": {
    "4": {
      "approval_threshold": 0.58,
      "min_ku_overlap_ratio": 0.2
    }
  }
}

================================================================================

FILE: csv_exports/course_syllabus.csv
--------------------------------------------------------------------------------
course_id,syllabus_text
1,"Algorithms, data structures, complexity, recursion, arrays, linked
lists, trees, graphs."
4,"Web technologies, HTML/CSS, JavaScript, basic databases, deployment
and web security basics."
12,"Network protocols, routing, switching, subnetting, transport and
application layers."
13,"Operating system design, process scheduling, memory management,
file systems, virtualization."
17,"Database models, SQL queries, normalization, indexing,
transactions, data integrity."
23,"Cybersecurity principles, threat modeling, vulnerabilities,
cryptography, ethics, defense."
24,"Internet threats, secure communication, encryption,
authentication, web application hardening."

================================================================================

FILE: tests/test_matcher.py
--------------------------------------------------------------------------------
from engine.matcher import compute_course_match


def test_matcher_equivalent_for_same_ku_and_title():
    source = {"course_name": "Operating Systems", "course_code": "CSIA
317", "credits": 3}
    target = {"course_name": "Operating Systems", "course_code": "CST
317", "credits": 3}

    result = compute_course_match(
        source_course=source,
        target_course=target,
```

```
        source_ku_ids={7, 12, 24},
        target_ku_ids={7, 12, 24},
        source_syllabus="scheduling memory file systems",
        target_syllabus="memory scheduling file systems",
    )
    assert result.decision == "Equivalent"
    assert result.ku_overlap_count == 3
    assert result.score >= 0.55


def test_matcher_not_equivalent_when_no_ku_overlap():
    source = {"course_name": "Data Structures", "course_code":
"CS101", "credits": 4}
    target = {"course_name": "Art History", "course_code": "AH101",
"credits": 3}

    result = compute_course_match(
        source_course=source,
        target_course=target,
        source_ku_ids={1},
        target_ku_ids={99},
    )
    assert result.decision == "Not Equivalent"

==========================================================================
==========

FILE: tests/test_pipeline.py
--------------------------------------------------------------------------
----------
import pandas as pd

from engine.data_loader import DataBundle
from engine.pipeline import match_transcript_to_institution


def test_pipeline_returns_ranked_matches():
    courses = pd.DataFrame(
        [
            {"course_id": 1, "institution_id": 1, "program_id": 1,
"course_name": "Operating Systems", "course_code": "CSIA 317",
"credits": 3},
            {"course_id": 2, "institution_id": 4, "program_id": 3,
"course_name": "Operating Systems", "course_code": "IT317", "credits":
3},
            {"course_id": 3, "institution_id": 4, "program_id": 3,
"course_name": "Web Development", "course_code": "SD102", "credits":
3},
        ]
    )
```

```
    bundle = DataBundle(
        courses=courses,
        course_ku=pd.DataFrame([{"course_id": 1, "ku_id": 12},
{"course_id": 2, "ku_id": 12}, {"course_id": 3, "ku_id": 4}]),
        knowledge_units=pd.DataFrame([{"ku_id": 12}]),
        institutions=pd.DataFrame([{"institution_id": 1},
{"institution_id": 4}]),
        programs=pd.DataFrame([{"program_id": 1}, {"program_id": 3}]),
        transfer_requests=pd.DataFrame(),
        students=pd.DataFrame(),
        course_syllabus=pd.DataFrame(
            [
                {"course_id": 1, "syllabus_text": "memory scheduling
processes"},
                {"course_id": 2, "syllabus_text": "processes
scheduling memory"},
                {"course_id": 3, "syllabus_text": "html css
javascript"},
            ]
        ),
    )
    transcript_data = {
        "classes": [{"course": "CSIA 317", "title": "Operating
Systems", "grade": "A", "credits": 3, "term": "Fall 2025"}]
    }

    result = match_transcript_to_institution(
        transcript_data=transcript_data,
        target_institution_id=4,
        data_bundle=bundle,
        top_k=2,
    )

    assert len(result) == 2
    top = result.iloc[0]
    assert top["target_course_code"] == "IT317"

================================================================================
==========

FILE: tests/conftest.py
--------------------------------------------------------------------------------
----------
from __future__ import annotations

import sys
from pathlib import Path


PROJECT_ROOT = Path(__file__).resolve().parents[1]
```

```
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

================================================================================

FILE: .gitignore
--------------------------------------------------------------------------------
.venv/
__pycache__/
.pytest_cache/
*.pyc
*.pyo
*.pyd

frontend/node_modules/
frontend/dist/

backend/.env
frontend/.env

================================================================================

FILE: requirements.txt
--------------------------------------------------------------------------------
pdfplumber
pandas
pytest

================================================================================

FILE: README.md
--------------------------------------------------------------------------------
i # Transfer Credit Match - Data Science Source Package

This package provides a medium-level backend/data-science workflow for
transfer credit matching:

- parse transcript PDFs
- map courses to Knowledge Units (KUs)
- score source courses against destination institution courses (KU +
title + credits + syllabus text)
- run transfer request workflows (submit, review, approve/deny)
- validate CSV data integrity
- export student transfer reports
```

- build a local SQLite database from seeded CSV files
- run tests for matcher and pipeline behavior

Project Layout

- `TranscriptPDF_reader/pdf_reader.py`: transcript parser
- `csv_exports/`: seeded institutions/programs/courses/KU datasets
- `engine/data_loader.py`: loads the CSV data bundle
- `engine/matcher.py`: KU + title + credit + syllabus scoring logic
- `engine/pipeline.py`: transcript-to-institution matching pipeline
- `engine/workflow.py`: submit + approve/deny workflow updates
- `engine/config.py`: institution-specific matching rule resolution
- `scripts/run_transfer_match.py`: main CLI for matching
- `scripts/build_sqlite.py`: creates `transfer_credit_match.db` from CSVs
- `scripts/validate_data.py`: validates key ID relationships across CSVs
- `scripts/workflow_cli.py`: submit/review/report CLI
- `config/matching_rules.json`: global + per-institution thresholds/weights
- `csv_exports/course_syllabus.csv`: optional syllabus snippets for similarity score
- `tests/`: starter automated tests

Setup

```bash
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Full Website Stack (Frontend + Backend)

You now have complete website code in:

- `backend/` -> FastAPI server
- `frontend/` -> React + Vite app

1) Start Backend API

```bash
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python run_api.py
```

Backend runs on `http://localhost:8000`

Useful endpoints:

- `GET /health`
- `GET /api/institutions`
- `GET /api/courses?institution_id=4`
- `POST /api/match/transcript` (multipart file upload)
- `POST /api/ai/explain` (API key-ready stub)
- `GET /api/ml/status`
- `POST /api/ml/train`
- `POST /api/ml/predict`
- `POST /api/ml/train-nlp`
- `POST /api/ml/predict-nlp`
- `POST /api/ml/analyze-kus`

2) Start Frontend Website

```bash
cd frontend
cp .env.example .env
npm install
npm run dev
```

Frontend runs on `http://localhost:5173`

API Key (Add Later)

When you are ready, add your key in `backend/.env`:

```env
OPENAI_API_KEY=your_key_here
```

The `/api/ai/explain` route is already wired to detect the key, so you can plug in real provider logic later without changing frontend wiring.

Your Custom ML Engine (Integrated)

Your provided script is now integrated at:

- `ml/transfer_matcher_full.py`

You can run it directly:

```bash
python ml/transfer_matcher_full.py train
python ml/transfer_matcher_full.py status
```

```
python ml/transfer_matcher_full.py predict '{"source":
[3,0,0,0],"target":[3,0,0,0]}'
python ml/transfer_matcher_full.py train_nlp --csv-dir csv_exports
python ml/transfer_matcher_full.py predict_nlp --source 1 --target 4
--csv-dir csv_exports
python ml/transfer_matcher_full.py analyze_kus --source 1 --target 4
--csv-dir csv_exports
```

And through the backend API:

```bash
curl -X POST http://localhost:8000/api/ml/predict \
  -H "Content-Type: application/json" \
  -d '{"source":[3,0,0,0],"target":[3,0,0,0]}'
```

Build Local SQLite Database

```bash
python scripts/build_sqlite.py --csv-root csv_exports --db-path
transfer_credit_match.db
```

Validate Dataset Integrity

```bash
python scripts/validate_data.py --csv-root csv_exports
```

Run Matching from a Transcript PDF

```bash
python scripts/run_transfer_match.py \
  --pdf "/absolute/path/to/transcript.pdf" \
  --csv-root csv_exports \
  --target-institution-id 4 \
  --top-k 3 \
  --threshold 0.55 \
  --rules config/matching_rules.json \
  --output match_output.csv
```

Output Columns

- `source_course_code`, `source_course_name`
- `target_course_code`, `target_course_name`
- `ku_overlap_count`, `ku_overlap_ratio`
- `title_similarity`
- `syllabus_similarity`

- `source_credits`, `target_credits`
- `score`
- `decision` (`Equivalent`, `Review`, `Not Equivalent`)

Scoring Rule (Configurable)

Default weighted score:

`score = 0.60 * ku_overlap_ratio + 0.20 * title_similarity + 0.10 * credit_ratio + 0.10 * syllabus_similarity`

- KU overlap is the strongest signal.
- Title, credit, and syllabus alignment refine confidence.
- You can tune thresholds and weights in `config/matching_rules.json`.

Workflow Commands (No Website Needed)

Submit a transfer request and create a pending match:

```bash
python scripts/workflow_cli.py submit \
  --csv-root csv_exports \
  --student-id 1 \
  --institution-from 1 \
  --institution-to 4 \
  --course-from 1 \
  --course-to 4
```

Director review (approve or deny):

```bash
python scripts/workflow_cli.py review \
  --csv-root csv_exports \
  --match-id 1 \
  --changed-by 1 \
  --decision Approved
```

Export a student transfer report:

```bash
python scripts/workflow_cli.py report \
  --csv-root csv_exports \
  --student-id 1 \
  --output student_transfer_report.csv
```

Testing

```bash
pytest -q
```

==========

FILE: api

New Files Produced

Backend / API (New)

- `backend/app/main.py` - Python (FastAPI routes)
- `backend/app/config.py` - Python (environment/config settings)
- `backend/app/services.py` - Python (transcript matching service layer)
- `backend/app/ml_service.py` - Python (ML integration service layer)
- `backend/app/schemas.py` - Python (Pydantic request/response schemas)
- `backend/run_api.py` - Python (API server entrypoint)
- `backend/requirements.txt` - Text (Python dependencies)
- `backend/.env.example` - Env file (environment variable template)

Frontend (New)

- `frontend/src/App.jsx` - JavaScript React (main UI page logic)
- `frontend/src/main.jsx` - JavaScript React (frontend entrypoint)
- `frontend/src/components/MatchTable.jsx` - JavaScript React (results table component)
- `frontend/src/styles.css` - CSS (UI styling)
- `frontend/index.html` - HTML (root page shell)
- `frontend/vite.config.js` - JavaScript (Vite build/dev config)
- `frontend/package.json` - JSON (Node scripts and dependencies)
- `frontend/.env.example` - Env file (frontend variable template)

==========